

SARKARAI OS — MASTER PRODUCTION PRD + SYSTEM ARCHITECTURE PROMPT

PROJECT TITLE

SarkarAI OS

“Autonomous AI Workflow Operating System for Indian Governance”

A futuristic AI-powered governance operating system for Indian citizen services, departmental workflows, approvals, grievance handling, document verification, and autonomous multi-agent orchestration.

The platform must behave like a REAL enterprise-grade AI SaaS platform — not a static demo.

This system is designed specifically for:

- Indian governance workflows
- Smart city administration
- District-level digital governance
- Lucknow municipal and administrative ecosystem
- AI-assisted public service automation

The entire product should feel:

- alive
- autonomous
- intelligent
- multilingual
- realtime
- cinematic
- futuristic
- enterprise-grade

This is a hackathon-winning AI Operating System.

CORE PRODUCT VISION

Indian governance systems suffer from:

- manual paperwork
- workflow delays
- multi-department bottlenecks
- document fraud
- poor citizen communication

- multilingual barriers
- non-transparent approvals

SarkarAI OS solves this using:

- autonomous AI agents
- realtime orchestration
- AI workflow routing
- OCR + verification
- multilingual AI assistants
- live analytics
- smart approvals
- AI-powered governance intelligence

The system must look like:

- Linear
- Palantir
- Vercel
- Notion AI
- Government Operating System
- futuristic Indian governance platform

IMPORTANT — REAL WORKING SYSTEM REQUIREMENTS

SarkarAI OS MUST NOT be:

- static UI
- fake dashboard
- placeholder app
- frontend-only project

EVERYTHING must FULLY WORK.

The platform must include:

- real backend execution
- real APIs
- real websocket updates
- real workflow execution
- real AI orchestration
- real database persistence
- real OCR extraction
- real RAG responses
- real notifications
- real document processing

Every:

- button
- workflow
- API
- graph
- chat
- upload
- animation
- dashboard action

must function properly end-to-end.

REAL USER EXPERIENCE

A judge should be able to:

1. Open landing page
2. Signup/Login
3. Upload documents
4. Trigger workflows
5. Watch AI agents execute LIVE
6. See realtime approvals
7. Receive notifications
8. Chat with AI assistant
9. Monitor orchestration graph
10. View analytics dashboards
11. Download generated reports
12. Switch languages instantly
13. Interact with AI voice assistant
14. See realtime backend responses

The platform should feel like:

“India’s Future AI Governance Operating System”

TARGET LOCATION CONTEXT

The platform is optimized for:

- Lucknow
- Uttar Pradesh
- Indian governance systems

Use:

- Lucknow city references
- local governance examples
- Nagar Nigam workflows
- Awadhi + Hindi + Hinglish support
- UP government style process flows

Example use cases:

- Gomti Nagar property registration
- Hazratganj grievance complaints
- Indira Nagar pension verification
- Nagar Nigam road repair complaints
- municipal approvals
- district office workflow routing

MULTILINGUAL SYSTEM

The platform MUST support:

- English
- Hindi
- Awadhi
- Hinglish

Users should be able to:

- switch languages globally
- use multilingual chatbot
- receive multilingual notifications
- talk to voice AI assistant

Example:

Citizen: “मेरी property registration कहाँ तक पहुँची?”

AI:

- current workflow stage
- pending approvals
- estimated completion
- missing documents
- officer remarks

Awadhi Example: “हमार registration काहे रुका बा?”

The AI assistant must respond naturally.

AI VOICE AGENT

Add a FULL AI VOICE AGENT.

Features:

- speech-to-text
- text-to-speech
- multilingual responses
- realtime voice interaction
- Gemini-powered conversational AI

Voice Assistant should:

- guide citizens
- explain workflows
- answer questions
- read approvals
- explain missing documents
- provide realtime status

Voice assistant must work in:

- Hindi
- English
- Awadhi
- Hinglish

UI should show:

- glowing AI orb
- waveform animations
- live transcript
- speaking indicators

FULL WEBSITE STRUCTURE

Create a COMPLETE production-grade website.

PUBLIC WEBSITE PAGES

1. Home Page

Ultra cinematic landing page.

Must include:

- hero section
- AI orchestration animation
- glowing network graph
- animated governance map
- workflow visualizations
- live dashboard preview
- animated metrics
- multilingual banners
- AI assistant demo
- realtime stats
- futuristic government visuals

Sections:

- Hero
- Features
- AI Agents
- Governance Problems Solved
- Realtime Workflow Demo
- Analytics Preview
- Testimonials
- FAQ
- Contact
- Footer

Add:

- floating gradients
- glassmorphism
- neon UI
- interactive animations
- dark/light mode toggle

2. About Page

Explain:

- vision
- mission
- governance modernization
- AI agent architecture
- multilingual governance
- Lucknow smart governance transformation

Include:

- architecture diagrams
- workflow lifecycle

- governance AI explanations
-

3. AI Chat Section

Dedicated AI assistant interface.

Features:

- Gemini chatbot
 - multilingual chat
 - voice AI
 - workflow query support
 - document summarization
 - RAG retrieval
 - realtime status responses
-

4. Services Page

List all governance modules:

- property registration
 - pension workflows
 - grievance redressal
 - municipal automation
 - document verification
 - smart approvals
 - fraud detection
 - analytics
-

5. Live Orchestration Page

MOST IMPORTANT PAGE.

Must visually show:

- AI agents activating
- node execution
- task routing
- approvals
- workflow pipelines
- live status updates

Use:

- React Flow
- animated edges

- glowing nodes
- websocket updates
- realtime execution

This page should WOW judges instantly.

USER DASHBOARDS

Citizen Dashboard

Features:

- uploaded applications
 - workflow stages
 - realtime tracking
 - notifications
 - AI chatbot
 - downloadable reports
 - multilingual support
 - voice assistant
 - approval history
-

Officer Dashboard

Features:

- assigned cases
 - pending approvals
 - AI recommendations
 - fraud alerts
 - workflow timelines
 - document viewer
 - approval actions
 - escalation alerts
-

Admin Dashboard

Features:

- workflow heatmaps
- department performance
- bottleneck analytics
- live agent monitoring
- realtime orchestration metrics
- system logs

- approval analytics
 - fraud detection reports
-

REALTIME EXECUTION REQUIREMENTS

When workflows start:

System must:

- activate agents automatically
- execute tasks live
- show realtime updates
- animate node execution
- display logs
- show task progress
- update approval chains dynamically

Use:

- WebSockets
- Socket.IO
- Redis Pub/Sub
- realtime event streaming

Frontend must update LIVE.

FULL AI AGENT SYSTEM

Create production-grade multi-agent architecture.

1. Planner Agent

Responsibilities:

- break workflows into tasks
 - assign tasks
 - manage orchestration
 - coordinate agents
-

2. Verification Agent

Responsibilities:

- OCR extraction

- Aadhaar parsing
 - PAN parsing
 - field validation
 - fake document detection
 - ownership validation
-

3. Rule Engine Agent

Responsibilities:

- eligibility validation
 - government policy rules
 - pension checks
 - workflow compliance
-

4. Translation Agent

Responsibilities:

- Hindi ↔ English
 - Awadhi ↔ Hindi
 - Hinglish understanding
-

5. Notification Agent

Responsibilities:

- realtime alerts
 - SMS
 - WhatsApp
 - email
 - dashboard notifications
-

6. Escalation Agent

Responsibilities:

- detect delays
 - fraud detection
 - suspicious claims
 - escalation routing
-

7. Audit Agent

Responsibilities:

- maintain logs
 - workflow explainability
 - traceability
 - compliance reports
-

GOVERNMENT PROBLEMS SOLVED

The website must contain detailed sections explaining:

1. Property Registration

Features:

- Aadhaar verification
- ownership validation
- stamp duty calculation
- missing document detection
- AI approvals
- summary generation

Citizen Flow: Upload → OCR → Verification → Approval → Registration Summary

2. Pension Disbursement

Features:

- eligibility checks
 - duplicate detection
 - scheme matching
 - bank verification
 - approval automation
-

3. Jansunwai / Grievance Redressal

Features:

- complaint classification
- department routing
- escalation handling
- realtime tracking

Example: Road issue → Nagar Nigam routing

4. Government Document Verification

Features:

- OCR extraction
- fake detection
- Aadhaar/PAN parsing
- field validation

Supports:

- PDF
 - JPG
 - PNG
 - scanned documents
-

5. Multi-Department Approval Automation

Workflow: Citizen → Verification → Revenue Dept → Approval → Certificate

6. Fraud Detection

AI detects:

- duplicate claims
 - suspicious uploads
 - workflow anomalies
 - delayed approvals
-

RAG SYSTEM REQUIREMENTS

Implement FULL RAG architecture.

Use:

- Gemini embeddings
- Vector database
- semantic retrieval
- multilingual indexing

The RAG system should store:

- governance FAQs
- citizen questions
- policies
- workflow documentation
- department SOPs
- pension rules
- property registration rules
- grievance procedures

The AI assistant must answer using retrieved governance context.

VECTOR DATABASE REQUIREMENTS

Use:

- pgvector OR Pinecone OR Weaviate

Store:

- embeddings
- citizen interactions
- workflow history
- multilingual knowledge
- government policies

Realtime retrieval should support:

- Hindi
 - English
 - Awadhi
-

FAQ SYSTEM

Add a HUGE FAQ section.

Include:

- property registration issues
- pension delays
- missing documents
- workflow tracking
- officer approvals
- grievance routing
- municipal complaints
- document upload issues

Each FAQ must include:

- problem
 - AI explanation
 - workflow solution
 - estimated resolution
-

SEO REQUIREMENTS

Implement FULL SEO optimization.

Include:

- metadata
- OpenGraph
- schema markup
- structured data
- multilingual SEO
- sitemap
- robots.txt
- dynamic metadata
- canonical URLs

Target keywords:

- AI governance platform
 - smart governance India
 - Lucknow AI governance
 - AI workflow automation
 - government AI operating system
 - multilingual AI governance
 - digital governance platform
-

UI/UX DESIGN SYSTEM

Theme:

- futuristic SaaS
- enterprise AI OS
- premium dashboards

Use:

- glassmorphism
- dark mode
- light mode
- gradients

- neon glow
- animated backgrounds

Fonts:

- Inter
- IBM Plex Sans

Animations:

- Framer Motion
 - live transitions
 - workflow animations
 - AI activity pulses
-

DARK/LIGHT MODE

Implement:

- dynamic theme switching
- persistent preferences
- smooth transitions

Dark mode should feel:

- cinematic
 - futuristic
 - glowing AI OS
-

BACKEND REQUIREMENTS

Backend must FULLY WORK.

Use:

- FastAPI
- PostgreSQL
- Redis
- Socket.IO
- LangGraph
- Gemini API
- OCR pipeline

Backend execution MUST happen server-side.

AI agents should NOT run on frontend.

Frontend should remain:

- clean
- responsive
- realtime

Backend should handle:

- orchestration
- queues
- AI execution
- OCR
- RAG
- approvals
- analytics

API REQUIREMENTS

Create FULLY WORKING APIs for:

- authentication
- workflow creation
- file uploads
- OCR extraction
- chatbot
- notifications
- approvals
- analytics
- workflow tracking
- orchestration monitoring
- voice assistant

All APIs must:

- validate input
- return proper responses
- support realtime updates
- include error handling

DATABASE REQUIREMENTS

Persist:

- users
- workflows
- approvals
- agent states

- OCR results
- notifications
- audit logs
- analytics
- workflow history
- chatbot memory

Use:

- PostgreSQL
- SQLAlchemy ORM
- Alembic migrations

DOCUMENT PROCESSING REQUIREMENTS

Uploads:

- PDF
- PNG
- JPG
- scanned files

Pipeline: 1. Upload 2. OCR extraction 3. Field parsing 4. Validation 5. Fraud detection 6. Workflow generation 7. Summary generation

Show REAL outputs.

No fake placeholders.

DEMO MODE

Environment Variable: DEMO_MODE=true

When enabled:

- preload workflows
- autoplay orchestration
- generate approvals
- simulate departments
- create live AI activity
- preload analytics
- auto-generate notifications

Perfect for judges.

VISUAL WOW FACTOR

The platform must feel:

- alive
- autonomous
- intelligent
- futuristic

Use:

- animated dashboards
 - glowing graphs
 - realtime node execution
 - floating metrics
 - cinematic UI
 - orchestration animations
-

TECH STACK

Frontend

- Next.js 15
- TypeScript
- Tailwind CSS
- Shadcn UI
- Framer Motion
- React Flow

Backend

- FastAPI
- Python
- PostgreSQL
- Redis
- Celery

AI

- Gemini 2.5 Pro ONLY
- Gemini function calling
- Gemini embeddings
- LangGraph

DO NOT USE OPENAI APIs.

AUTHENTICATION

Use:

- Clerk Auth

Roles:

- Citizen
- Officer
- Admin

Include:

- RBAC
 - JWT validation
 - secure routes
-

STORAGE

Use:

- Supabase Storage

Store:

- documents
 - workflow reports
 - OCR outputs
 - analytics exports
-

DEPLOYMENT

Frontend:

- Vercel

Backend:

- Railway

Database:

- Supabase PostgreSQL

Realtime:

- Redis + Socket.IO
-

PROJECT STRUCTURE

apps/ web/ api/

packages/ agents/ ui/ db/ lib/ workflows/

REQUIRED OUTPUTS

Generate FULL production-grade code for:

- frontend
- backend
- database
- AI agents
- APIs
- websocket layer
- OCR pipeline
- LangGraph orchestration
- RAG system
- analytics
- dashboards
- voice AI
- deployment
- Docker setup
- CI/CD
- README
- demo workflows
- seed data

NO pseudo-code.

ONLY real executable production-grade code.

HOMEPAGE CONTENT REQUIREMENTS

Homepage must include:

- live governance statistics
- AI orchestration preview
- multilingual banners

- Lucknow smart governance messaging
 - glowing workflow graph
 - AI voice assistant CTA
 - governance AI showcase
 - realtime metrics
-

FOOTER REQUIREMENTS

Footer must include:

- About
- Services
- Governance Modules
- AI Agent System
- Contact
- Social Links
- Privacy Policy
- Terms
- Support
- Language Switcher

Add:

- Lucknow smart governance branding
 - multilingual footer navigation
-

SECURITY REQUIREMENTS

Implement:

- rate limiting
 - JWT auth
 - secure uploads
 - RBAC
 - audit logs
 - API validation
 - encryption
 - secure websocket handling
-

FINAL EXPERIENCE

The final system should feel like:

“The AI Operating System for Future Indian Governance.”

Judges should feel:

- shocked
- impressed
- immersed
- convinced this is real

The platform must feel:

- production-grade
- enterprise-ready
- futuristic
- autonomous
- visually unforgettable

Generate EVERYTHING step-by-step starting from: 1. project setup 2. architecture 3. folder structure 4. frontend 5. backend 6. database 7. AI orchestration 8. APIs 9. deployment 10. demo mode 11. live orchestration 12. multilingual AI system 13. voice assistant 14. RAG pipelines 15. realtime analytics

END GOAL:

“A complete autonomous AI governance operating system for India.”